Examen 1: CNN para censura en video

Ramírez Trejo César Uriel
Terrazas López Mariana

1 de noviembre de 2025

Sección 1

Introducción

En este trabajo se implementa un programa que permite blurear automáticamente contenido no apto para todo público en elementos audiovisuales utilizando una red neuronal convolucional entrenanda con Tensorflow.

Sección 2

Marco teórico

Las redes neuronales convolucionales son una arquitectura especialmente utilizada en la detección local y multiescala de objetos en imagenes y reconstrucciones 3D. La habilidad de este tipo de algoritmos recae en la tarea de las convoluciones que fungen como extractoras de características seguidas de una red neuronal completamente conexa.

2.1 Última capa de la red.

En una red neuronal de este tipo la función de activación de la última capa de la red juega un rol fundamental. Básicamente, tenemos dos activaciones importantes:

- Función sigmoide: Si utilizamos la función sigmoide habremos detectado el nivel de activación de cada neurona de la capa final por separado. Esta es la idea central de la **clasificación multicategoría**, la cual permite detectar la presencia de múltiples objetos en una misma imagen.

- Función softmax: Si se hace uso de esta función, entonces obtenemos una distribución de probabilidad con las activaciones de las neuronas de las últimas capas, lo cual nos es el fundamento téorico de la **clasificación monocategoría**, cuando deseamos encontrar cual de los objetos estudiados se encuentra con mayor probabilidad en una imagen.

2.2 Entrenamiento por mini-lotes

Un **mini-lote** de datos es un subconjunto propio de nuestro conjunto de entrenamiento. Cuando entrenamos una red neuronal no actualizamos el valor de los parámetros, en vez de eso, formamos mini-lotes A_i que verifican que

$$L = {}_k A_k \text{ y } A_i \cap A_j = \emptyset \text{ para } i \neq j$$

se suele utilizar mini-lotes en vez de Corremos el modelo sobre cada dato del mini-lote y promediamos el valor de la función de pérdida obtenida en cada iteración, es decir,

$$p = \frac{1}{n} \sum_{i}^{n} f(\mathrm{x}_i).$$

Con este valor de error promedio actualizamos el valor de nuestros parámetros

$$\mathrm{x} = \mathrm{x} - \eta \nabla(p).$$

Tensorflow llama **paso** o **step** a cada actualización de parámetros por cada mini-lote, de hecho, podemos calcular el número de pasos simplemente como T/t donde T representa la cantidad de instancias de nuestra base de datos y t representa el tamaño del mini-lote que elijamos.

Debemos señalar que un valor más bajo de lote implica hacer más *pasos* en el entrenamiento aumentando la complejidad del algoritmo, se ha demostrado que es benefico utilizar como tamaños de lote potencias de 2.

2.3 Batch normalization

Retomando lo mencionado en la subsección anterior, la normalización juega un rol central dentro del entrenamiento de modelos de Inteligencia Artificial, reduciendo la complejidad algóritmica temporal enormemente. La *normalización por lotes* es un tipo de normalización que toma en cuenta el mini-lote completo al momento de su aplicación.

Para esto consideremos un conjunto de k imagenes RGB de tamaño $m \times n$ (en el peor de los casos posible tras preprocesamiento). Ahora definamos $\mu_{i,j,c}$ a la media muestral del conjunto de todos los pixeles de las k imagenes en la posición i, j y del canal c. De manera análoga, podemos definir a $\sigma_{i,j,c}$. Después de esto aplicamos la ecuación de normalización

$$I^{\alpha}_{i,j,c} = \frac{I^{\alpha}_{i,j,c} - \mu_{i,j,c}}{\sigma_{i,j,c}}.$$

donde

- α es el número de imagen de las k imagenes.
- i es el número de fila dentro de la imagen.
- j es el número de columna dentro de la imagen.
- $$\mu_{i,j,c} = \frac{1}{k}\sum_{\alpha=1}^{k} I^{\alpha}_{i,j,c}$$
  para $i \in \{1, 2, \dots, m\}$, $j \in \{1, 2, \dots, n\}$ y $c \in \{1, 2, 3\}$.
- $$\sigma_{i,j,c} = \sqrt{\frac{1}{k}\sum_{\alpha=1}^{k}\left(I^{\alpha}_{i,j,c} - \mu_{i,j,c}\right)^2}$$
  para $i \in \{1, 2, \dots, m\}$, $j \in \{1, 2, \dots, n\}$ y $c \in \{1, 2, 3\}$.

De esta manera obtenemos nuevas imagenes con valores de color más pequeños que aceleran el entrenamiento del modelo.

2.4 Dropout.

Este método de regularización consiste en "apagar" aleatoriamente las neuronas de una capa determinada con la finalidad de que el modelo no genere una dependencia fuerte hacia ninguna de las neuronas que la compone. Matemáticamente, el modelo coloca el valor más bajo posible (según sea el caso) en la neurona eliminando su efecto sobre el resultado final.

2.5 Early Stopping

Este regularizador es el encargado de detectar cuando la métrica medida del modelo deja de progresar y entonces detiene el modelo con la finalidad de evitar el sobreajuste o una baja mayor en la aptitud del modelo.

2.6 Transfer Learning

El aprendizaje por transferencia es una técnica que aprovecha los pesos y representaciones aprendidas por un modelo entrenado en una tarea fuente para aplicarlos a una tarea diferente. Esta estrategia es útil cuando ambas tareas comparten características de bajo nivel (por ejemplo, bordes, texturas o patrones espaciales) pero difieren en el dominio o en las clases objetivo.

2.7 Fine-Tuning

El también llamado ajuste fino es una técnica específica dentro del *Transfer Learning* que consiste en ajustar parcialmente los pesos del modelo preentrenado para adaptarlo a la nueva tarea. En lugar de usar la red congelada, se reentrenan ciertas capas (generalmente las superiores) con una tasa de aprendizaje reducida.

El proceso típico de **Fine-Tuning** consta de los siguientes pasos:

1. Cargar un modelo preentrenado: Se utiliza una red como MobileNet, ResNet o EfficientNet, entrenada en un conjunto grande (por ejemplo, ImageNet).
2. Modificar la capa de salida: Se reemplaza la última capa densa para adaptarla al número de clases de la nueva tarea.
3. Congelar las primeras capas: Las capas iniciales permanecen fijas, pues contienen características generales.
4. Entrenar las capas superiores: Sólo se reentrenan las últimas capas (más específicas) para aprender las características del nuevo dominio.

Sección 3

Desarrollo

3.1 Dataset utilizado

Con la finalidad de evitar sesgos se ha realizado la compilación de secciones de datasets abiertos logrando así una variedad de los objetos buscados en diferentes contextos con nuestro modelo. Nuestro dataset final consta de poco menos de **1,200 imagenes** de distintas fuentes, respetando la siguiente proporción:

- Mitad de datos con y sin categoría: Esto pretende de lograr que el modelo sea capaz de encontrar alguna categoría o no por igual.
- Clases balanceadas dentro de las categorías: Esto hace que en probabilidad una vez que se ha detectado algo sea igualmente posible encontrar cualquiera de las categorías buscadas.

Aunque no parecería de utilidad muchos conjuntos de datos completamente diferentes pueden contener imagenes que nos fueron de utilidad. Por ejemplo, el conjunto sobre detección de perros. De manera precisa, la distribución de los datos entre e intra categorías así como las fuentes de donde fueron extraídas se describe en el siguiente diagrama de pastel

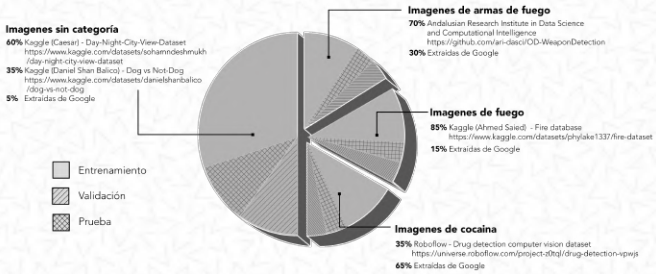


Figura 1: Composición del dataset. El dataset completo consta de menos de 1,200 imagenes extraídas de diferentes fuentes. Creación propia

3.2 Método de blureo

En este trabajo utilizamos una convolución como método de blureo, concretamente un filtro de suavizado conocido como **filtro de medias** el cual consta simplemente de una matriz con 1 en todas sus posiciones multiplicado por el inverso del producto entre la cantidad de filas y columnas, concretamente en nuestro caso $K \in \mathbb{R}^{51} \times \mathbb{R}$ dado por $K_{i,j} = 1/51^2$. El método puede visualizarse en la siguiente imagen.

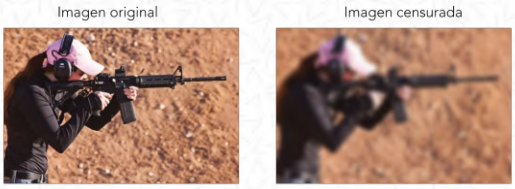


Figura 2: En este trabajo se utilizó una convolución de medias para blurear las imagenes en las que se identificó contenido no apto para todo público. Creación propia

3.3 Arquitectura MobileNet

MobileNet es una familia de arquitecturas de redes neuronales convolucionales diseñadas para dispositivos con recursos limitados (móviles, embebidos). La característica central es la reducción drástica del cómputo y parámetros manteniendo una precisión razonable, usando principalmente *depthwise separable convolutions* y optimizaciones arquitectónicas posteriores.

En la siguiente imagen podemos visualizar el diagrama de flujo que describe la relación entre los métodos implementados, comenzaremos ahora a hablar brevemente de cada uno.

3.4 Algoritmo de búsqueda

Dentro de una imagen podemos encontrar el objeto con diferentes tamaños, sin embargo, nuestro objetivo es buscar la presencia de un objeto a cualquier escala. Con esto en mente, se realizó una partición de subimagenes donde buscaremos una a una dentro de ellas. Llamamos a estas imagenes **parches** y bastará con encontrar en una sola de ellas para decir que la categoría está presente en la imagen y censurarla.

Cuando buscamos el objeto en toda la imagen asumimos que el objeto central de la fotografía es el objeto en sí, llamamos a esto **búsqueda global**. Por otro lado, cuando buscamos en secciones más pequeñas nos aseguramos de buscar el objeto en muchas secciones de la imagen, llamamos a esto **búsqueda local**.

De manera empírica podemos observar que el tamaño de los parches utilizados está intimamente relacionada con el desempeño del modelo.

En nuestro desarrollo utilizaremos tres parches simultaneamente.

- Parches de 750×750: Parches pequeños para búsqueda exhaustiva por secciones, solo aceptaremos que está activada cuando el valor de activación sea muy alto evitando así malas activaciones.
- Parches de 1024×1024: Medianos que nos ayudan a buscar la presencia global de los objetos buscados. Pediremos un nivel de confianza medio para asegurar la presencia de los objetos
- Parche de 2048×2048: Generalmente un solo parche, lo cual busca únicamente sobre toda la imagen, para asegurarnos de activar pedimos un nivel bajo de confianza.

Para mezclar los resultados de las tres búsquedas diremos que una clase α se encuentra en una imagen I cuando la presencia de la categoría está en algún parche de ambas búsquedas exhaustivas (garantizando su presencia en búsquedas de diferente escala y realizando una búsqueda local) o cuando se encuentra de manera general en la imagen completa (búsqueda global).

Para reducir la complejidad de la búsqueda utilizamos *threading* con tres hilos (uno para cada tamaño de

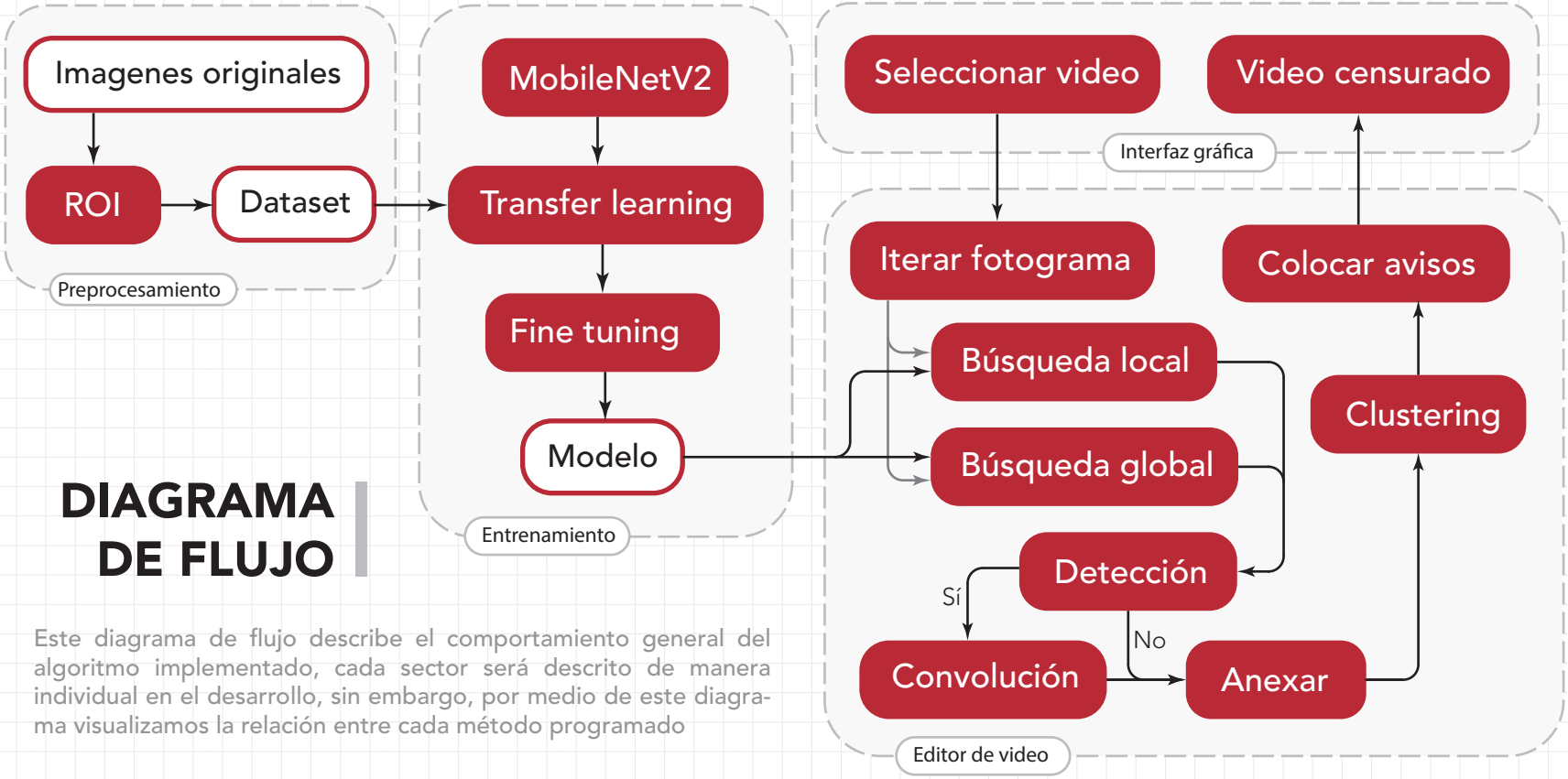


DIAGRAMA DE FLUJO

Este diagrama de flujo describe el comportamiento general del algoritmo implementado, cada sector será descrito de manera individual en el desarrollo, sin embargo, por medio de este diagrama visualizamos la relación entre cada método programado

parches) realizando los métodos de búsqueda simultaneamente.



Figura 3: Búsqueda local de fuego en imagen. Creación propia



Figura 4: Búsqueda local de fuego en imagen. Creación propia

3.5 Selección de imagenes analizadas

Comencemos esta parte recordando un hecho de los filtros *busca bordes*. El método intrinseco que utilizan es el hecho de que los objetos regularmente tienen secciones completas del mismo material con tonalidades similares, lo cual hace que detectar el cambio en la tonalidad sea equivalente a detectar el borde de los objetos en una imagen.

En el caso de los videos sucede algo similar. Los videos en una perspectiva simplista pueden pensarse como fotografías dispuestas a lo largo del tiempo. Estudios han demostrado que para que el cerebro humano asocie el cambio en los fotogramas con movimiento es necesario al menos reproducir 24 fotogramas por segundo.

Además, es claro que para que nuestro ojo sea capaz de captar una escena, los elementos principales de esta deben permanecer por varios fotogramas seguidos, con esto en mente, es claro que habra múltiples fotogramas con escenas similares y por ende objetos en ellas similares y justo esa es la idea central para reducir el procesamiento de nuestro algoritmo: En vez de revisar fotograma a fotograma, simplemente revisaremos cada n fotogramas y si el k–ésimo fotograma debe blurearse se bloquean todos los $n - 2$ fotogramas siguientes, es decir los fotogramas $k+1, k+2, \ldots, k+n-1$ revisando así muchos menos fotogramas y logrando buenos resultados con un menor tiempo de procesamiento. Al valor n que acabamos de describir le denominamos **paso**.



Figura 5: Detección modular en fotogramas, paso de 3 fotogramas. Como podemos ver, esta técnica no es precisa pero las escenas no blureadas correctamente generalmente son imperceptibles al ojo humano. Imagenes de fotograma generadas con IA.

3.6 Clustering de notificaciones

Se implementó una notificación sobre el video que avisa de una escena no apta para todo público antes de que ocurra. Imaginemos que existen 2 fotogramas revisados seguidos con escenas que deben blurearse, entonces la notificación se superpondría, lo cual ocasionaría problemas de legibilidad, para evitarlo debemos crear grupos de clusters que permiten definir en que tiempo mostrar adecuadamente las notificaciones.

Para realizarlo, simplemente consideremos los tiempos donde aparece cada clase (que nos ofrece el modelo) y posteriormente "anexemoslas" dentro de un único arreglo, posteriormente, con la duración d del tiempo que tarda una notificación desde que comienza a aparecer a desaparecer (en nuestro caso 3.5 segundos) verifiquemos que el tiempo que existe entre una detección y otra sea mayor a d, si no es así pertenece al mismo cluster, si lo cumple se crea un nuevo cluster.

3.7 Arquitectura implementada

El modelo desarrollado se basó en la arquitectura MobileNetV2 preentrenada con el conjunto de datos *ImageNet*, empleando la técnica de *transfer learning* y posteriormente un ajuste fino (*fine tuning*). Las imágenes fueron cargadas desde carpetas de entrenamiento y validación, redimensionadas a 128 × 128 píxeles y procesadas en lotes de 32 muestras.

Durante la primera etapa, el modelo base se

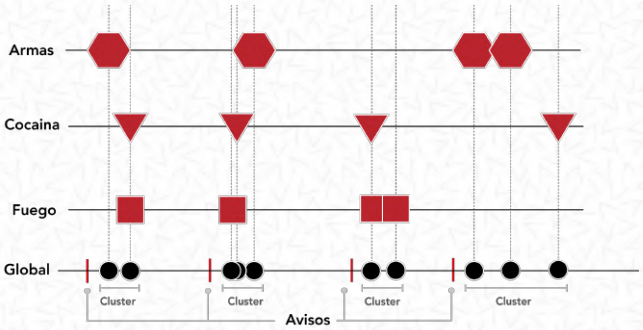


Figura 6: Grupos de detecciones y localización de notificaciones en el tiempo. Creación propia

mantuvo congelado y se añadieron capas densas personalizadas conformadas por una operación de *GlobalAveragePooling2D*, seguida de capas *Dropout* para evitar sobreajuste, una capa densa de 128 neuronas con activación *ReLU* y una capa de salida con activación *sigmoid*. El entrenamiento inicial se realizó con el optimizador *Adam*.

En la segunda fase, se aplicó *fine tuning* descongelando las últimas 20 capas de la MobileNetV2, reduciendo la tasa de aprendizaje a 1×10^{-5}. Se emplearon los *callbacks EarlyStopping* y *ReduceLROnPlateau* para optimizar el proceso de entrenamiento.

Sección 4

Resultados

El modelo alcanzó una precisión del 92 %, mostrando un desempeño sólido en la clasificación de las imágenes evaluadas. Es importante destacar que el rendimiento del modelo depende en gran medida de la paleta de colores presente en los videos de entrada, ya que las variaciones de iluminación y tonalidad pueden influir en la extracción de características durante el proceso de inferencia. Para validar su funcionamiento, el modelo fue aplicado a un video de prueba en el cual se observa un desempeño visual satisfactorio, con una correcta identificación de las clases y una respuesta coherente ante los cambios de color y textura del entorno.

Sección 5

Conclusiones

Este método resulta muy eficaz para su propósito reduciendo el tiempo laborar que toma blurear un video manualmente, en trabajos futuros se sugieren las siguientes mejoras:



Figura 7: Imagenes blureadas con el programa.

- Segmentación en el blureo que permita visualizar la escena casi por completa exceptuando únicamente aquellas zonas de la imagen donde se encuentren los objetos no deseados.
- Paralelización de procesos para poder implementarlo en tiempo real, para ello el tiempo de procesamiento de 1 segundo de video debe tardar menos de 1 segundo de procesamiento.
- Uso de una red neuronal más robusta que la arquitectura MobileNet.
- Aumento en la cantidad y variedad de imagenes que componen el conjunto de entrenamiento.

Referencias

[1] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. Obra fundamental sobre redes neuronales y aprendizaje profundo. MIT Press, 2016. URL: `https://www.deeplearningbook.org`.

[2] Yann LeCun, Yoshua Bengio y Geoffrey Hinton. "Deep Learning". En: *Nature* 521 (2015). Revisión general del aprendizaje profundo y sus aplicaciones en visión por computadora., págs. 436-444. DOI: `10.1038/nature14539`.